Ясишена Діана Зіядівна, група К28. Програмний проєкт 4, варіант 20.

Умова. Структура даних з двох цілих полів (m = 2). Частоти операцій: поле 0 - read 10%, write 5%; поле 1 - read 50%, write 5%; string - 30%.

Схема захисту. Використано два м'ютекси std::mutex: mutex0 захищає поле 0, mutex1 захищає поле 1. Операції read і write блокують тільки м'ютекс свого поля. Операція string блокує обидва м'ютекси (завжди спочатку mutex0, потім mutex1), копіює значення полів, відпускає м'ютекси і вже потім складає рядок.

Чому так: з полем 1 працює 55% операцій, з полем 0 - 15%, тому окремі м'ютекси дозволяють діям з різними полями не чекати одна на одну. Для string потрібні обидва поля, тому вона бере обидва м'ютекси. Обидва м'ютекси бере тільки string і завжди в одному порядку, тому взаємоблокування неможливе. shared_mutex не використано: операції дуже короткі, а shared_mutex блокується повільніше, ніж звичайний mutex. Під час перевірки варіант із shared_mutex працював на 30–40% повільніше.

Результати. Час у мілісекундах, середнє з 10 запусків. Усього 600 000 операцій; при 2 і 3 потоках кожен потік має свій файл з половиною або третиною операцій. Час читання файлів не враховано. Режим Release (O3), CLion, macOS.

файл	1 потік	2 потоки	3 потоки
а) частоти за умовою	19.13	25.73	38.75
б) усі частоти рівні (по 20%)	17.19	22.05	35.14
в) read 5/5%, write 40/40%, string 10%	12.04	16.53	25.56

Висновки. Результати відповідають очікуванням. Що більше потоків, то довше працює програма: операції дуже короткі, тому потоки більше чекають один на одного біля м'ютексів, ніж працюють паралельно. Файл в) найшвидший, а а) найповільніший, бо найдовша операція - string, а її в а) 30%, у б) 20%, у в) лише 10%.

Що зроблено самостійно. Код написала самостійно, ШІ використовувала лише для перевірки правильності.